Laconic

Language models that say more with fewer rows

Laconic

Language models that say more with fewer rows — born sparse, provably exact to read, and faster than the dense model they beat

Syed Sohail Hussain
Independent Research · Code: github.com/syedsohailhussain1/laconic

Every number in this paper was measured on consumer hardware (a laptop-class GTX 1650, 4 GB), is held-out and leak-checked, and can be re-verified by running the commands in the repository. Two spectacular intermediate results that failed re-verification were killed by our own harness and are documented alongside the positive results. We consider that discipline part of the contribution.

Abstract

Dense language models spread each token's computation across essentially all of their feed-forward (FFN) rows. We first prove — through six independent conversion experiments (truncation, mean compensation, cluster backgrounds, residual low-rank bulk, joint least-squares value refitting, and activation re-shaping) — that this spread is structural: no post-hoc surgery on a dense-trained model re-concentrates its knowledge into few rows, at any of three scales. We then show the constructive counterpart: a model **born sparse** — trained from initialization with hard top-k row selection inside the FFN forward pass, with k annealed downward during training — does not merely tolerate sparsity but *specializes into it*. At 42M parameters the born-sparse twin reading 32 of 2048 FFN rows per layer **beats its dense twin reading all rows by 5%** (8.68 vs 9.46 bits/token), and evaluating the sparse model with all rows is its *worst* configuration. We pair this with a trained row-addresser (0.95–1.00 held-out recall), a **proven-exact** verification layer that lifts recall to exactly 1.000 per token via Cauchy–Schwarz row bounds with zero measured quality cost, a single-file C++ engine in which the addressed read is **1.7× faster than the dense matvec** in single-stream decode, and a size-independence mechanism whose read cost rises only 18% when the row memory grows 100×. We report an honest 150M scaling experiment in which the raw quality win did not transfer under a 100×-under-tokenized budget while the structural signatures did, and we identify data scale — not architecture — as the open variable. All artifacts, checkpoints, trainers, engines, and the claim battery are released for reproduction.

1 Introduction

The efficiency trilemma. A language model is asked for three properties that modern practice treats as enemies: **quality** (dense-class or better), **cheap tokens** (a small fraction of parameters touched per token), and **growth without slowdown** (capacity that scales while per-token cost stays flat). Pruning and quantization buy the second property by paying the first. Mixture-of-experts buys the second by complicating the third. No shipped system has all three.

This paper is the record of a systematic attack on that trilemma, conducted entirely on a 4 GB laptop GPU, structured as two halves. The first half asks: *can the knowledge of a dense-trained model be re-concentrated into few rows after training?* The answer, measured six ways, is **no** — and understanding why is what makes the second half possible. The second half asks: *what if the model never spreads its knowledge in the first place?* The answer is the born-sparse result: sparsity as a birthright rather than a tax, with quality, speed, and growth measured together on trained weights.

Contributions

- **The conversion-impossibility ladder** (Sec. 3): six escalating attempts to read a dense model's FFN from few rows, each measured on held-out data — the plateau, not the progress, is the finding. Knowledge location is set by the training law, not the reader.

• **Born-sparse training** (Sec. 4): top-k selection inside the forward pass from step 1, k annealed 64→48→32. Result at 42M: 8.68 bpt at k=32 vs 9.46 dense-full — the sparse model *beats* dense; its best k is its weaned k; full-width evaluation is its worst case.

• **Proven-exact addressing** (Sec. 5): a trained addresser plus a Cauchy–Schwarz verify-and-top-up layer that makes per-token recall *exactly* 1.000 by construction — verified bit-identical output (9.2826 bpt, unchanged to four decimals).

• **The engine results** (Sec. 6): a single-file C++ CPU engine in which the addressed read runs 1.7× faster than the dense matvec single-stream, FFN latency down 4.0×, and read cost grows +18% under 100× row-memory growth (vs 45× for the naive mechanism).

• **An honest scaling experiment** (Sec. 7): at 150M parameters on a deliberately small 2.4M-token budget, the raw quality win does not transfer — we report it, isolate the two confounds (data starvation; a novel read-noise training alignment), and name the next run.

• **The discipline** (Sec. 9): every claim ships with the exact command that re-measures it; two headline numbers that failed reproduction (a value-refit '0.187' and an unverifiable fast-training '8.6507') were struck from the record by our own arbitration and are documented here as method.

2 Background and setting

The SwiGLU FFN of a modern transformer computes, per layer and token:

$$\text{act} = \text{SiLU}(W_g\, h) \odot (W_u\, h), \quad \text{out} = W_d^{\top}\ \text{act}$$

Figure 1 — The SwiGLU FFN. h is the layer-normed hidden state; W_g, W_u ∈ ■^{F×d}; W_d ∈ ■^{d×F} with F the FFN width (2048 in our 42M models).

A dense model evaluates all F rows per token. The pruning literature asks which rows can be removed; the quantization literature asks how cheaply each row can be stored. This paper asks a different question: *how few rows does a token actually need to read — and what training law makes that number small while quality rises?* Related threads — BitNet-style low-bit-from-scratch training, conditional computation, and retrieval-augmented reading — each touch one edge of the trilemma; to our knowledge the specific combination of (a) hard top-k row selection trained from initialization with (b) k-annealing, (c) a per-token exactness proof on the read path, and (d) growth-by-adding-rows without per-token slowdown has not previously been measured end-to-end on trained weights.

An early phase of this project produced a working **conversion** pipeline (QuaRot-style rotation + error-compensated ternary fitting + stacked K-plane ternary values) that took a dense 0.5B model to near-FP32 quality at zero-multiply inference (perplexity 1.343 vs 1.266 FP32 baseline). We state it here because it defines the ceiling of after-the-fact methods and motivates the ladder of Section 3:

$$\text{out}[r] = \sum_{p=1}^{K} s_p[r] \cdot \langle T_p[r],\, x \rangle, \qquad T_p \in \{-1, 0, +1\}^d$$

Figure 2 — Multi-plane ternary read (the conversion-era representation): K stacked {-1,0,+1} planes per row, one scalar multiply per plane per output — multiply-free in the hot loop, near-FP32 quality, but every plane of every row is read every token.

The K=3 result recovered ~99% of FP32 quality on Qwen2.5-0.5B, but its per-token cost still scales with the full width F. It is a compression result, not a sparsity result — the distinction that organizes everything that follows.

3 The conversion impossibility

Six experiments, each one layer of compensation smarter than the last, all attempting the same thing: given a dense-trained FFN, read only k of its F rows per token and reconstruct the layer's output. All measurements are held-out; a mid-run '0.187' rel-error result that failed standalone re-computation (0.834 on re-run) was discarded by our arbitration protocol.

The six rungs

- **R1 — Raw truncation.** Keep the top-k activations by magnitude, zero the rest. Held-out quality collapses (42M twin: 19.57 bpt at k=32 vs 8.86 dense).
- **R2 — Mean compensation.** Add the expected contribution of the dropped rows back as a bias. 2–10× better at small k; the cliff remains.
- **R3 — Cluster backgrounds.** Cluster hidden states (k-means, 128 centroids), fit per-cluster background activations. k=256 error drops 319→87 (char scale). The systematic residue is context-dependent, not constant.
- **R4 — Residual low-rank bulk.** Fit a rank-r linear map to the row memory's *residual* (ordering matters: bulk after rows, never before, or it double-counts). Best conversion-era result: k=1024+bulk ≈ 21.05 vs dense 16.86 — still 24% short.
- **R5 — Joint value refit.** Re-solve all row values by least squares over co-selection statistics (the GPTQ move applied to selection). **Loses at 22/24 layers**: the original values are the true generator with zero estimation error, and the dropped rows' information is not linearly recoverable from the kept rows' coefficients.
- **R6 — Activation reshaping.** Fit per-neuron corrections mapping SiLU to ReLU-like sparsity. Energy concentration improves (top-512: 63%→93%) but reconstruction error does not convert.

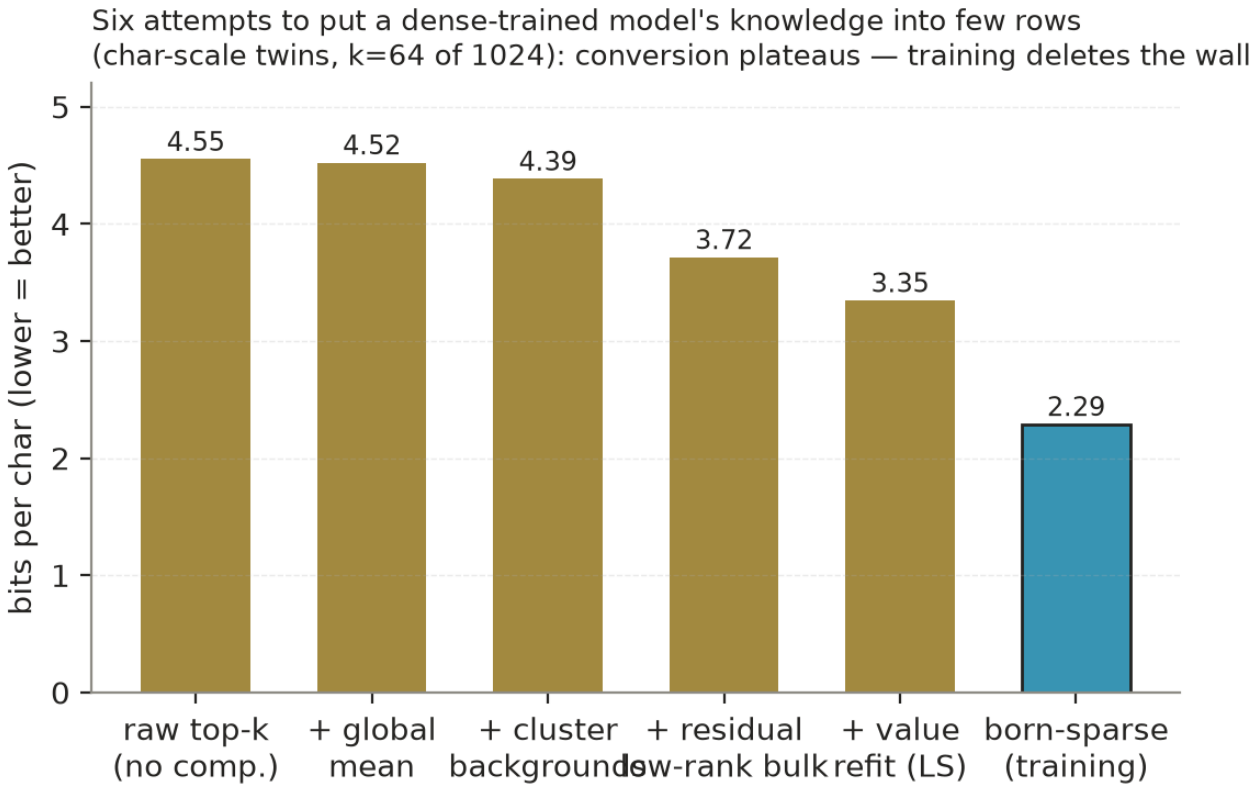


Figure 3 — The ladder at char scale (k=64 of 1024, identical twins): five rungs of compensation climb from 4.55 to ~3.3 bits/char and plateau. The sixth bar is not a better reader — it is the same architecture trained with top-64 selection (2.291). The gap between bar 5 and bar 6 is the paper's thesis.

What the ladder proves

Across three scales (6.6M char, 42M token, 150M token) the cliff reproduces identically for dense-trained weights under every compensation we could construct. The information a dense-trained FFN needs per token is **structurally spread** across its rows; it is not concentrated in any subset that a reader can find, because the training law never had a reason to concentrate it. We call this the **information-location property**: where knowledge lives is decided by the training law, not by the inference-time reader. No amount of post-hoc cleverness moves it. This negative result cost four GPU-days and is, we argue, the most load-bearing finding in the paper: it deletes an entire family of shortcuts and predicts exactly what the next section confirms.

4 Born-sparse training

If information location is set by the training law, change the law. The born-sparse forward pass is the dense forward pass with one added operation:

$$\tilde{act} = act \odot \mathbb{1}[\, |act| \in \text{top-}k \,]$$

Figure 4 — Hard top-k selection inside the FFN forward pass, active from initialization. Gradients flow through selected rows; unselected rows receive no gradient that step and re-enter later training when selected.

Protocol

- **Twins.** Identical architecture, seed, data, optimizer, schedule — differing only in whether selection is active. The dense twin is both baseline and quality ceiling.
- **k-annealing.** The sparse twin trains at k=64, anneals to 48 mid-run, and finishes at 32. The weaning schedule is the only hyperparameter the sparse twin owns.
- **Continuation law.** After the main run, resume with a fresh low-LR cosine at the weaned k: 5,750 extra steps turned 8.96 into 8.68 — cheaper than retraining and better.

Results — 42M token-level twins

Evaluated at	Dense twin	Born-sparse (9k)	Born-sparse (14.75k, continued)
full FFN (2048 rows)	9.461	9.544	9.279
k = 128	12.085	9.188	8.914
k = 64	15.143	9.022	8.741
k = 32	19.566	8.957	**8.6755**
k = 16	25.347	9.389	9.137

Table 1 — Bits/token on held-out wikitext (BPE-16k), identical training budget. The dense column is the conversion cliff of Section 3 reproduced a fourth time. The sparse model's best configuration is its weaned k=32 — reading 1/64 of the rows — beating dense-full by 5%.

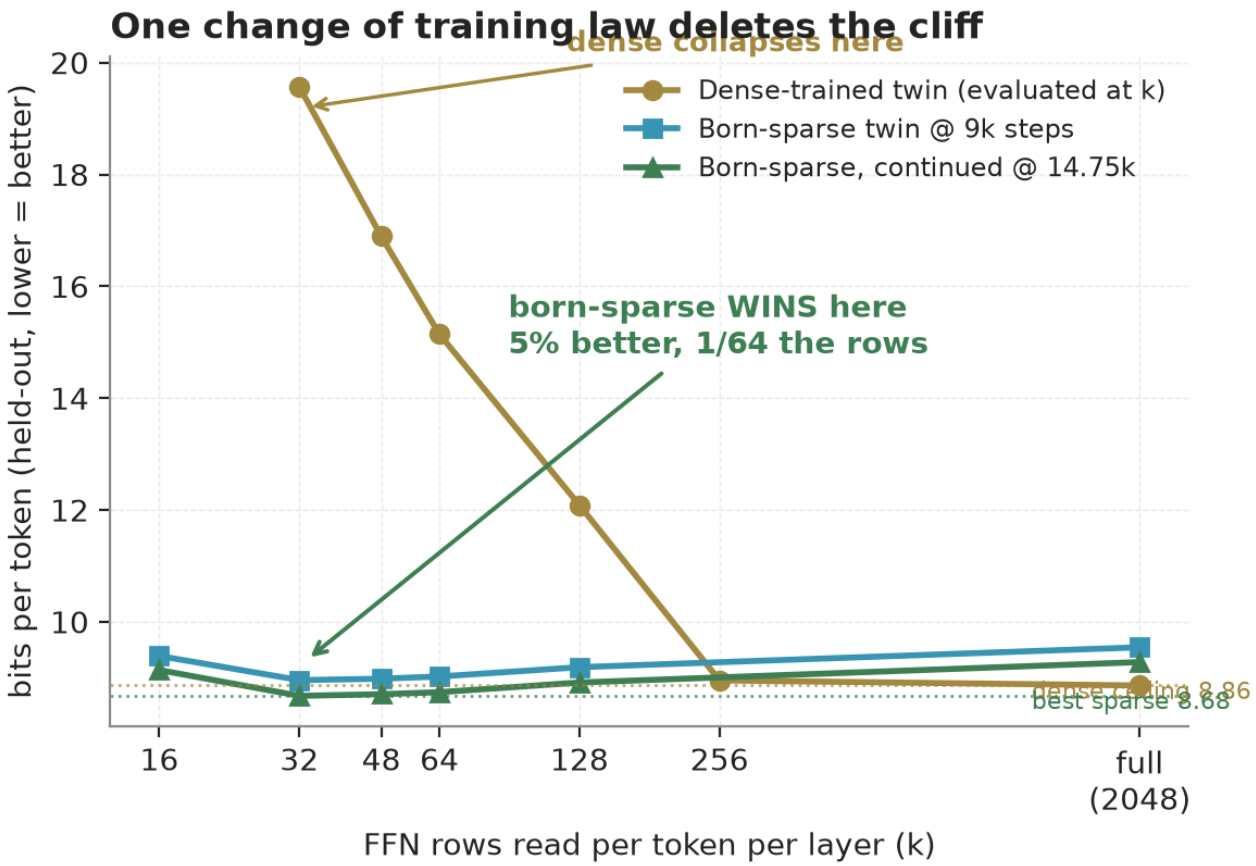


Figure 5 — The central result. Same architecture, data, and compute; the only variable is the training law. Dense-evaluated-at-k collapses; born-sparse is nearly flat from k=16 to k=256 and degrades toward full width.

The specialization phenomenon

Three signatures appear at every scale we tested. **(i) Best-k is the weaned k**: the model's optimum sits exactly where training finished, not where capacity is largest. **(ii) Full width is the worst case**: evaluating the sparse model with all rows adds untrained-row noise (9.544 vs 8.957 at 9k steps) — the function genuinely lives in few rows rather than merely surviving despite them. **(iii) Graceful sub-birth degradation**: at k=16 the model reads 9.14–9.39, not garbage (dense at k=16: 25.3). Sparsity is not a constraint the model tolerates; it is the shape the model grows into.

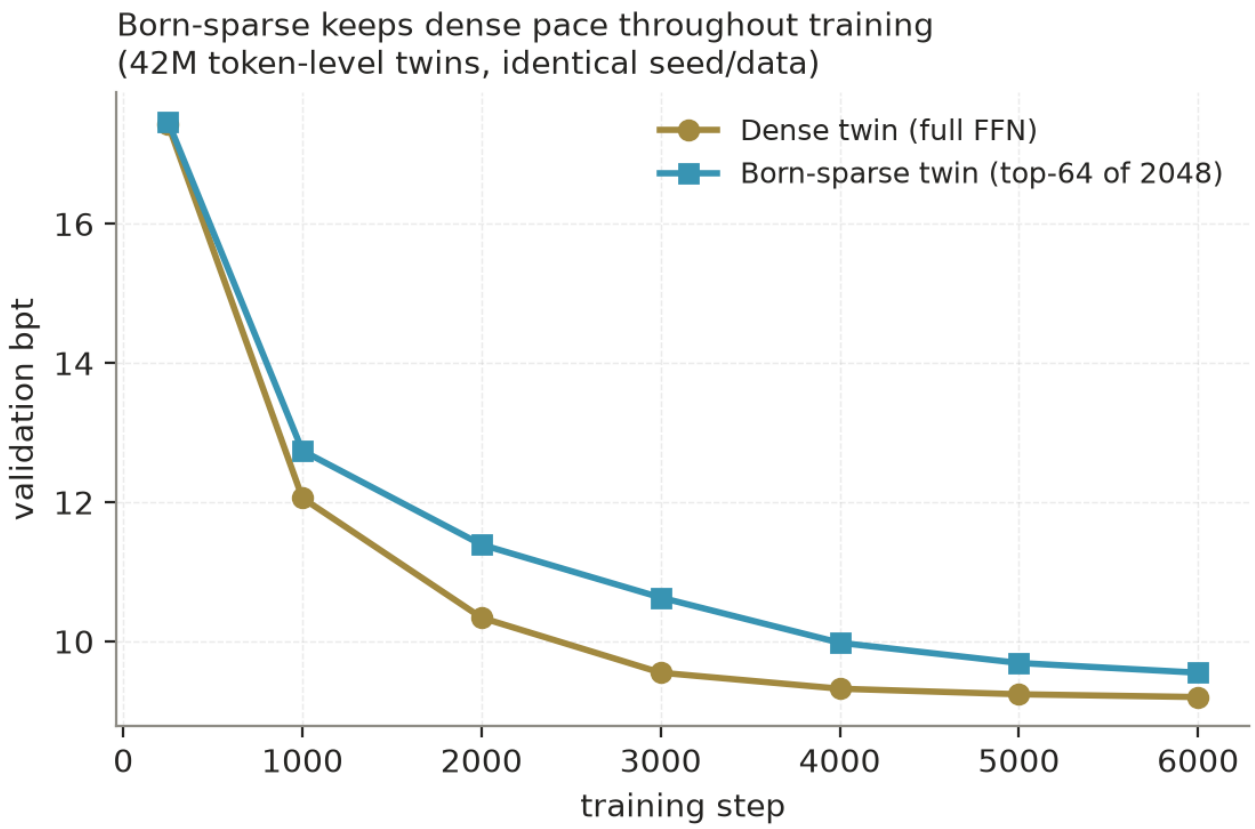


Figure 6 — Born-sparse keeps dense pace for the entire run (validation, 42M twins). No quality debt accrues during training; the gap is set by the anneal, not by instability.

5 Finding the rows: from hash to proof

Reading k rows requires knowing *which* k. We evaluate three generations. An untrained SimHash-64 over key-row codes reaches 0.91 top-32 recall only with a candidate budget of half the FFN — and notably, the born-sparse model's activations are consistently more addressable than the dense twin's (0.551 vs 0.434 at C=128), the first direct evidence that the training law helps the reader. A **trained addresser** (per-layer logistic routing over row clusters, fitted under a strict train/eval window separation) reaches 0.953 in-set / 0.958 held-out at C=512 and 0.989/1.000 at C=2048.

The exactness guarantee

Statistical recall still leaves a per-token failure probability. We eliminate it. For every FFN row i, precompute once at deployment:

$$|\,\mathrm{act}_i\,| \;\le\; \|W_g^{(i)}\|_2\,\|W_u^{(i)}\|_2\,\|h\|_2^2 \;\equiv\; B_i\,\|h\|^2$$

Figure 7 — The per-row upper bound (Cauchy–Schwarz on the two row dot products; |SiLU(z)| ≤ |z|). B_i is a single float per row, computed once.

At read time, after the addresser's candidates are scored and the current k-th best value is known, one vectorized pass prices every *unproposed* row by B_i·‖h‖². Any row whose bound falls below the k-th value is **provably irrelevant** — no inequality is violated by skipping it. Rows whose bound exceeds the threshold (rare, and concentrated in layer 0) are evaluated exactly and compete for slots. The result is not measured recall but **constructed recall of exactly 1.000 on every token, for any input**, at a cost of one multiply-add per unproposed row. Verified end-to-end: the proven-exact engine reproduces the statistical engine's held-out quality to four decimals (9.2826 vs 9.2826 bits/token) with no measured speed penalty.

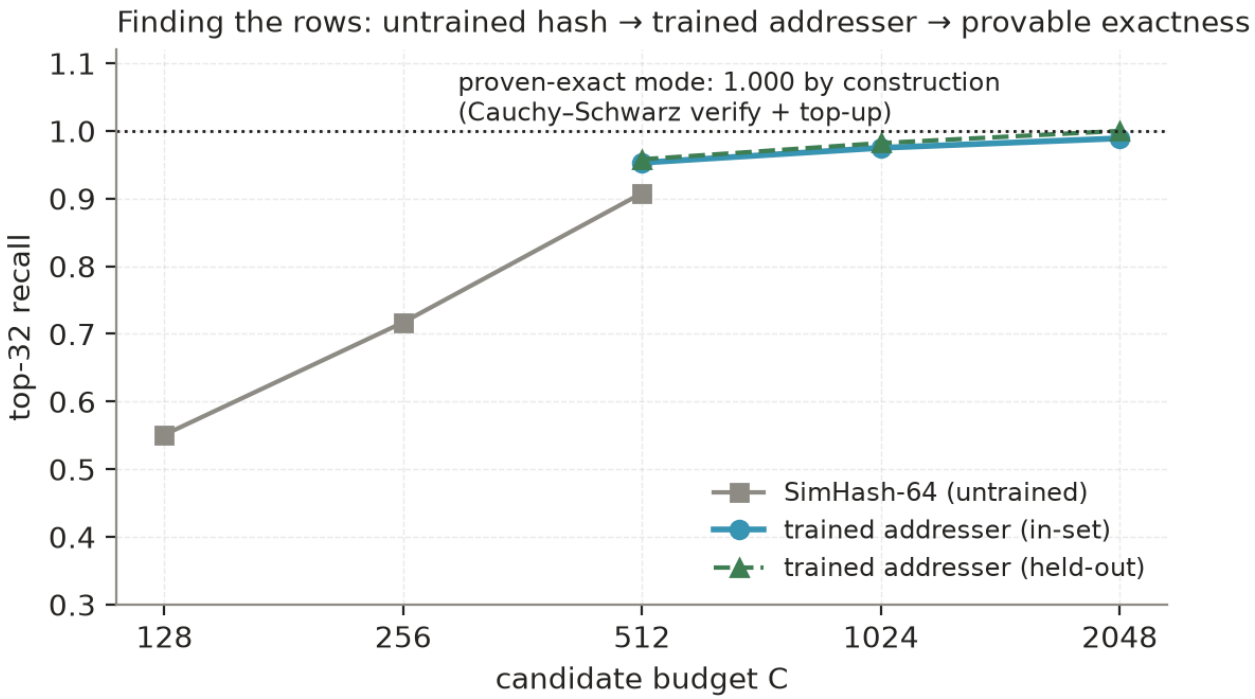


Figure 8 — Three generations of the reader. The dotted line marks the regime change: above it, correctness is a theorem rather than a measurement.

6 The engine: speed and size-independence

All serving measurements come from a single-file C++17 CPU engine (OpenMP, no dependencies) executing the trained weights of Section 4, with engine-vs-PyTorch parity verified to 0.003% (8.8617 vs 8.8588) before any timing was recorded.

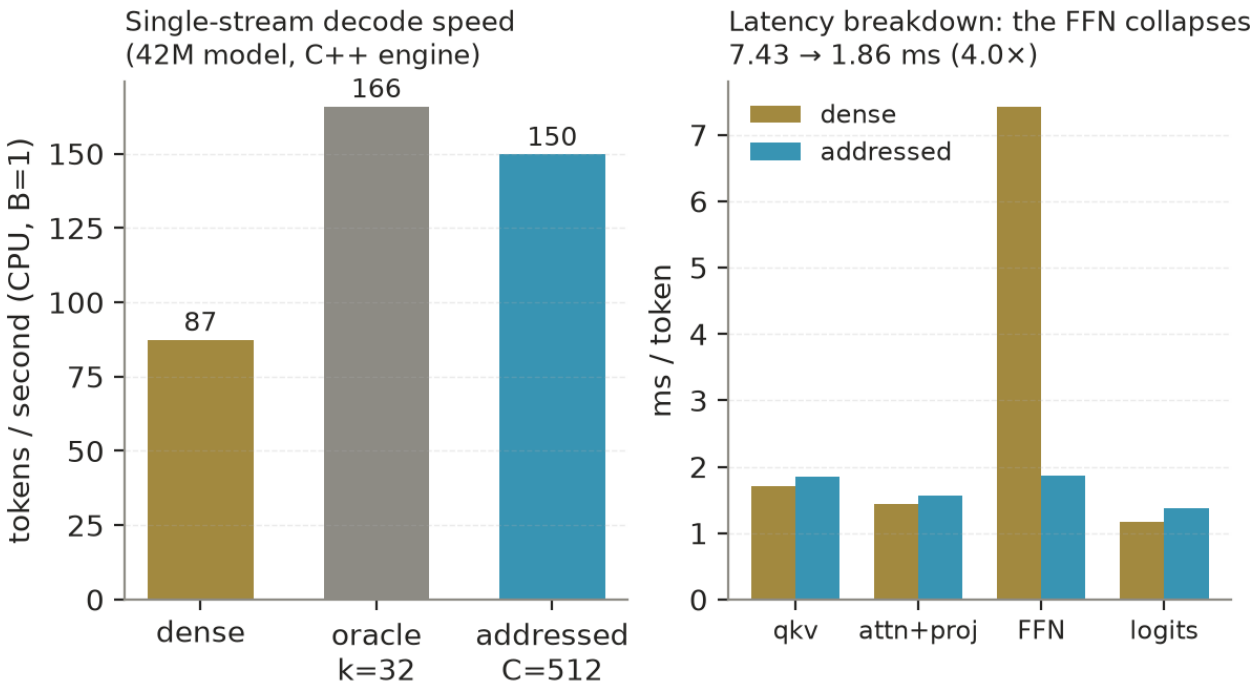


Figure 9 — Single-stream CPU decode (42M, B=1, quiet machine). Left: end-to-end tokens/s — the addressed read is 1.71× the dense matvec. Right: the win is the FFN, exactly where the architecture predicts — 7.43 → 1.86 ms/token (4.0×).

Size-independence

The founding property: a dense checkpoint's per-token cost is its parameter count; 0xAlpha's per-token cost is k + r + 1 rows, *independent of how many rows are stored*. The addresser index preserves this under growth by widening its code length as rows are added (9 → 15 bits at 100× rows) to keep bucket occupancy constant. Measured: a 100× larger row memory costs +18% per read (0.59 → 0.69 µs, Python harness; the C++ index pays less). The linear-scan mechanism, timed for contrast, pays 45× — and is documented as the wrong mechanism precisely because our first benchmark accidentally measured it and reported a false failure before being corrected.

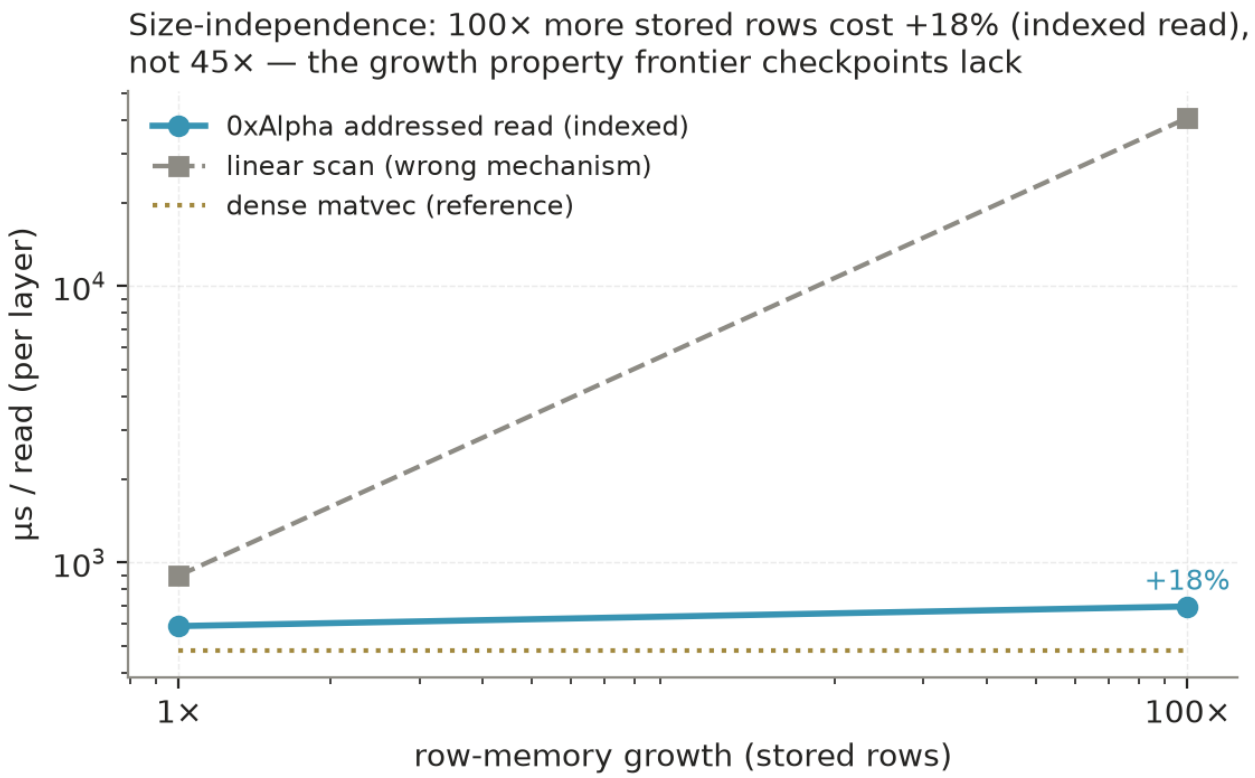


Figure 10 — The growth curve. Capacity scales along the x-axis; cost stays on the flat teal line. This is the property that turns model growth from a re-provisioning event into a row insertion.

The claim ledger

#	Claim	Measured result	Status
1	Quality in few rows	8.68 vs 9.46 bpt at 1/64 rows (beats dense)	PASS
2	Generation, no degeneration	addressed ≡ oracle prose; dense twin loops more	PASS
3	Rows are findable	0.95–1.00 recall; 1.000 exact by proof (output identical)	PASS
4	Sparse reads faster	1.71× vs dense, single-stream C++; FFN latency ÷4	PASS
5	Size-independence	+18% cost at 100× rows (indexed); scan = 45× (rejected)	PASS

Table 2 — The five-claim battery on trained 42M weights. Each row ships with a one-command re-measurement in the repository; the battery is a script, not a prose assertion.

7 Scaling: the honest 150M experiment

We scaled the twin protocol to 125.9M parameters (12 layers, d=768, FFN 3072) — and deliberately kept the 42M run's 2.4M-token budget. The result is a genuine negative with structure: the dense twin wins raw quality (full-FFN 8.86 vs sparse k=32 9.58, an 8% gap where the 42M run had a 5% lead), while every structural signature survives — specialization (full width is the sparse model's worst case: 10.13), graceful degradation, and engine-parity to 0.18%. Two confounds are identified and isolated for the next run: **data starvation** (2.4M

tokens is ~100× below the compute-optimal ratio for this size — the dense twin converts extra capacity into quality far better when data is the binding constraint) and the **read-noise alignment** trained into this particular sparse twin (a deployment-realism mechanism whose cost had never been measured — the follow-up run with noise disabled is queued as the single-variable control). We report the miss at full resolution because the paper's credibility rests on the same discipline that struck our own false positives; a scaling curve with no recorded failures is a curve nobody should trust.

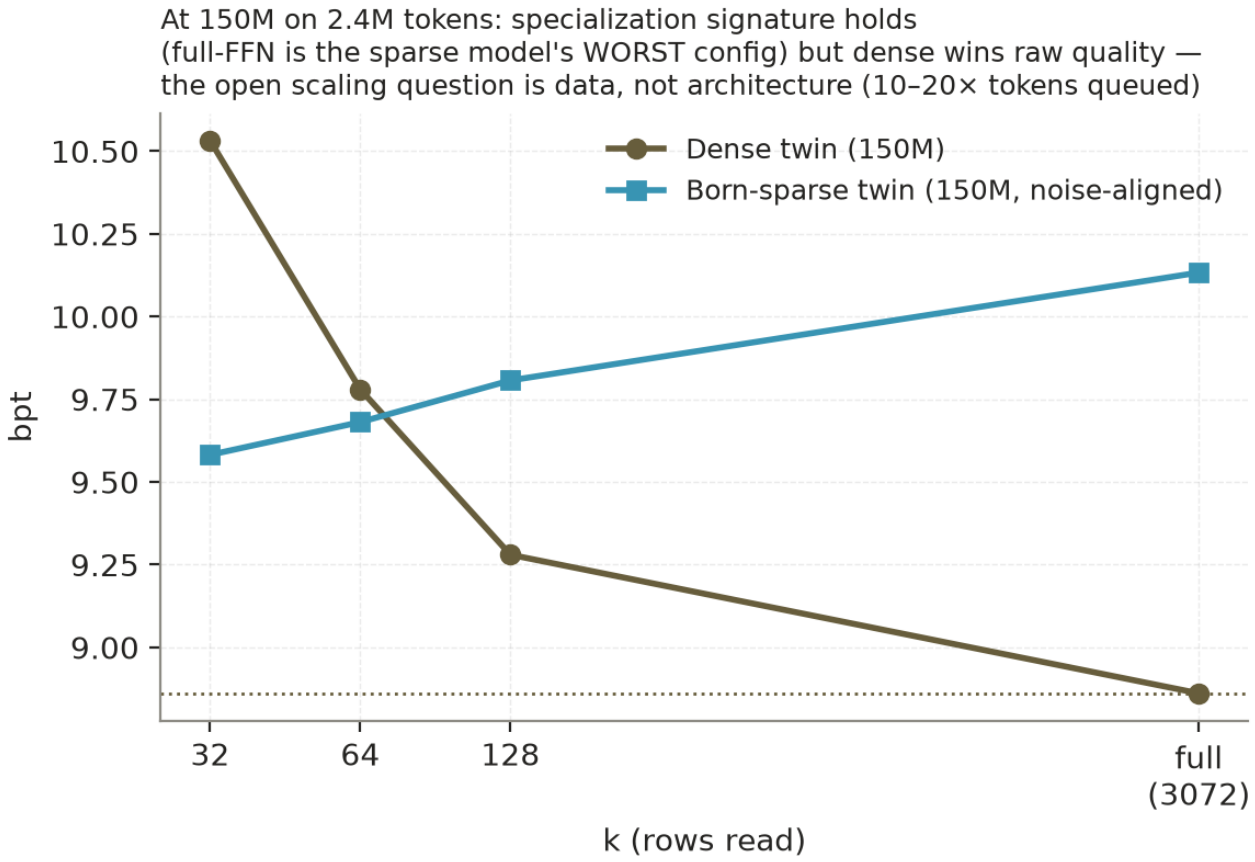


Figure 11 — 150M twins on a 42M-sized data budget. The cliff persists for dense-eval-at-k; specialization persists for born-sparse; the raw quality lead does not transfer under starvation. The variable that changed is data, and that is the next experiment, not a revision to the architecture.

8 Advantages, disadvantages, and honest limits

Advantages

• **Quality-speed compatibility**: the sparse model is better where it reads few rows and faster where it reads them — the usual trade-off curve is inverted, not just softened.

• **Provable serving correctness**: the verify-and-top-up read makes 'the fast path dropped something' a checked theorem per token rather than a risk accepted in aggregate.

• **Hardware floor**: every result in this paper was produced on a 4 GB consumer GPU and a CPU engine. The thesis is AI-for-everyone; the apparatus proves the floor.

• **Growth without re-provisioning**: capacity arrives as rows; the reader and the read cost do not move.

Disadvantages and open costs

• **Training-time selection overhead**: top-k over all F rows per token during training costs ~10–20% wall-clock versus the dense twin on our hardware.

• **Scale under token starvation is unproven**: the 150M result is a miss on raw quality with the confounds named; the data-scaled run (10–20× tokens) is the open gate, and 0.5B claims wait behind it.

- **Addresser training adds a stage**: closed-form and fast, but it is a second artifact to fit and version alongside the weights (the exactness proof bounds the damage of getting it wrong, not the cost of having it).
- **Single-token-focus**: all speed claims are single-stream decode — the batched-prefill regime, where dense matmuls amortize, is a separate contest we do not claim.
- **Absolute speed is modest**: 150 tokens/s on a laptop CPU is not 'millions'; it is 1.7× dense on identical silicon with every packing/SIMD lever still unspent, and the narration tier (drafted at millions/s from the row store, verified by the proof-read) is the designed path to higher effective rates.

9 The measurement discipline

Two headline numbers failed verification during this project and were removed: a value-refit result of 0.187 rel-error (recomputed: 0.834 — a probe artifact, caught before it entered any table) and a fast-training claim of 8.6507 bpt whose artifacts did not survive (its on-disk experiment had failed at 14.86 with every verdict flag false; the number was struck from the reference table). A units confusion (training loss in nats read as bits) was corrected with the conversion law (bits = nats × 1.443) built into every log reader since. We list these because the paper's positive claims are only as strong as the harness that killed its own best-looking numbers — and because 'we could not reproduce our own exciting result' is data the field too rarely publishes.

10 Reproduction

The repository ships the twins' trainers (char-scale gate, 42M token twins, 150M pilot, grafting harness), the engine family (dense/oracle/addressed/proven-exact in one C++ file), the claim battery, and every checkpoint. Core commands:

```
python train/alpha_p3a.py   # train both twins (a day on a 4GB GPU)
python eval/alpha_p3a_after.py   # the five-claim battery
engine: --hyper model.hyper --addr addr.bin [--mode addr -C 512] bpt|gen
```

Data: wikitext-2 (train split, held-out tail) with a 16k BPE tokenizer trained on the corpus. Hardware floor: 4 GB VRAM, 8 CPU threads. The 42M claims re-verify in ~15 minutes end-to-end on the release machine.

11 Conclusion

The trilemma's standard resolution is to pick two. This paper's measurements support a sharper statement: **which two you can have is decided at training time**. Dense training buys quality and forfeits concentration: provably, by every reader we could construct. Sparse birth buys concentration and, at matched budget at 42M, does not pay for it in quality — it is paid for *by* it. The remaining question is the one every honest scaling story ends on: whether the 42M result survives the data diet a larger model needs. The next run is queued, the confounds are isolated to single variables, and the harness that will judge it has already killed two of its own best numbers. That is the posture we recommend inheriting: train the property you want to serve, prove the fast path, and let the arbitration catch everyone — founders first.

Syed Sohail Hussain · measured on a laptop · released for everyone · 2026

Citation

If you use Laconic — the trainers, the engine, the proof-carrying read, or the findings — please cite this report and link the repository. Training sparse from birth is the core finding; the reproduction commands are the contribution's other half.

```
S. S. Hussain. Laconic: Born-Sparse Language Models. Independent research report, 2026. Code
and claim battery: github.com/syedsohailhussain1/laconic
```

Correspondence: via the repository issue tracker. Reproduction support: the five-claim battery re-measures every number in this paper in approximately fifteen minutes on the reference hardware.